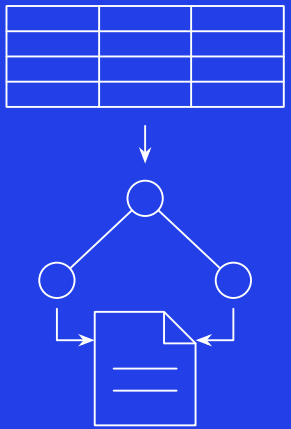
WEEK 1 / LAB 02

MLOps: Machine Learning Operations

Data Validation, Chronological Split & Baseline Tracking

Validate data. Compare models. Preserve evidence.



A teammate can inspect your baseline

Three core outcomes — not a promise of a better model or an extra platform.

1 **Protect data and evaluation boundaries**

Reject invalid rows, preserve the canonical CSV, use the fixed chronological split and keep the test period held.

2 **Compare two predictors fairly**

Fit the mean and fixed Random Forest on train only; measure both MAEs on identical validation rows.

3 **Retrieve and verify local evidence**

Find both actual runs, match identities and metrics, and reload the saved model. An unchanged-config repeat is optional stretch.

Complete the core before any stretch

Lab 2 uses the remaining Week 1 time, not a separate four-hour meeting.

1 **Required pipeline**

Validate, split, fit mean + fixed RF, retain two MLflow runs, save/reload, test and review. Keep data, features, split and config fixed.

2 **Bounded choice, not another investigation**

Fault A (missing column) is the worked example; your group designs one more fault of its own. Choose how to implement and test; justify that choice briefly.

3 **Optional repeat; no extra platform**

Repeat unchanged only after the core is complete; not required for m1 and no extra marks. No API, Docker, cloud, tuning or test-set scoring.

Common contract; your implementation choices

Use the approved Lab 2 extension in the existing repository. Change the Lab 1 lead author.

1 Supplied foundation

Canonical CSV/manifest, approved lock, CLI and pipeline scaffolds, public tests and PR checks. Do not build a platform from scratch.

2 Fixed boundaries

Preserve source bytes, feature order, date intervals, train-only fit, mean + fixed RF, validation MAE and required output identities.

3 Your decisions

Implement within the scaffold, choose focused tests and design your own fault.
A teammate in your group reviews the choice and evidence.

PART 1

Validate the data. Protect the split.

A believable score starts before the model is fitted.

CHECKPOINT A

Preserve source bytes

Enforce the contract

Reserve the test period

Start only from a verified, matching Lab 2 package

Commands run in the supplied bike-demand project root, not this slide workspace.

SETUP COMMANDS / EXAMPLE BRANCH NAME

```
uv sync --locked
uv run --locked python -m bike_demand.preflight
git switch -c lab02/data-baseline
```

Confirm the data, manifest and lock before changing code.

Missing package or setup blocked for five minutes? Ask for the supported recovery route.

Install MLflow before training

The matching Lab 2 lock must include MLflow; uv installs its approved version.

Missing MLflow? Request the approved Lab 2 package; do not use global pip.

BEFORE TRAINING / INSTALL + CHECK IN THE PROJECT ROOT

```
uv sync --locked
uv run --locked mlflow --version
```

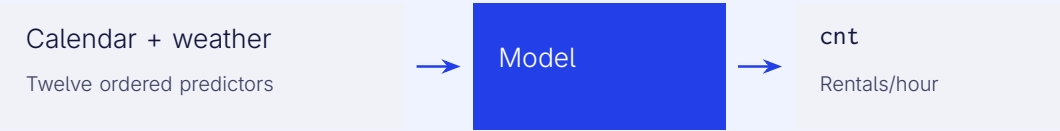
AFTER TRAINING / SECOND TERMINAL, SAME PROJECT ROOT

```
uv run --locked mlflow server \
  --backend-store-uri sqlite:///tracking/mlflow.db \
  --host 127.0.0.1 --port 5000
```

Open `http://127.0.0.1:5000`; select the experiment and compare both runs.

Estimate demand given calendar and weather inputs

This is conditional hourly demand – not a forecast of unknown future weather.



`cnt = casual + registered`

Both components reveal the answer. Keep them out of the feature matrix.

Keep exactly twelve predictors, in this order

Raw CSV fields and model inputs are different contracts.

1–4	`season`, `yr`, `mnth`, `hr`
5–8	`holiday`, `weekday`, `workingday`, `weathersit`
9–12	`temp`, `atemp`, `hum`, `windspeed`

Exclude `cnt`, `casual`, `registered`, `instant`, `dteday` from every feature matrix.

Reject invalid data before training starts

Parse declared types; name the failing field or rule. Never silently fill, clip or drop.

Categories	`season`: 1–4; `yr`, `holiday`, `workingday`: 0 or 1 `mnth`: 1–12; `hr`: 0–23; `weekday`: 0–6; `weathersit`: 1–4 All category values must be integers.
Weather values	`temp`, `atemp`, `hum`, `windspeed`: finite numbers in [0, 1].
Rows and schema	Required columns/values present; parseable dates. Unique positive integer `instant`; nonnegative integer counts.

Preserve `data/raw/hour.csv`; test invalid copies instead.

Train on the past; reserve the later test period

Course date intervals are inclusive. No shuffle and no random train/test split.

Train

01 Jan–31 Dec 2011

Fit mean and Random Forest

Validation

01 Jan–30 Jun 2012

Evaluate both

Test

01 Jul–31 Dec 2012

Held; not evaluated

Reject dates outside these intervals; do not silently discard rows.

Every row needs one partition and a stable order

Implement the split contract and focused tests before fitting either predictor.

1 **Sort by `(dteday, hr, instant)`**

Parse dates; use `instant` to break ties after date and hour.

2 **Assert nonempty, disjoint and complete membership**

Test inclusive boundaries; every canonical row belongs to exactly one interval.

3 **Record observed train/validation counts and end keys**

First and last `(dteday, hr, instant)` – not full-calendar hour totals.

PART 2

Fit two predictors. Keep their evidence.

Keep the same inputs and make both results identifiable.

CHECKPOINT B

Same validation rows

Two actual local runs

Save, reload and verify

A feature-dependent model needs a fair comparator

Both predictors fit on train only and are evaluated on the same validation rows.

NAIVE BASELINE

Training-label mean

$\widehat{y} = \overline{y}_{\text{train}}$ for every validation row

No calendar/weather dependence.
Never average validation or test labels.

FEATURE-DEPENDENT BASELINE

Random Forest

$\widehat{y} = f(X)$ using the twelve predictors

Fit on the training matrix only.
Keep the supplied configuration fixed.

Use the course configuration without tuning

These are proposed teaching defaults, not a measured accuracy claim.

PYTHON SETTINGS – COURSE DEFAULTS, NOT A MEASURED RESULT

```python
from sklearn.ensemble import RandomForestRegressor
model = RandomForestRegressor(
    n_estimators=50,
    max_depth=10,
    random_state=42,
    n_jobs=1,
)
```

Train rows only

Twelve ordered columns.
No extra feature search.

Other defaults

Use the locked library.
Do not change the config.

Measure absolute error on identical validation rows

Lower MAE is better for this fixed comparison; no target score is prescribed.

$$\mathrm{MAE} = \frac{1}{n_{\mathrm{val}}} \sum_{\text{validation rows}} |y - \widehat{y}|$$

Units: rentals/hour

Record both measured MAEs and explain the comparison.
Do not invent a win, a run ID or a passing threshold.

Commit training code before capturing its identity

First commit the completed code/config on your task branch; then validate and run.

EXPECTED STARTER COMMANDS – CAPTURE ACTUAL OUTCOMES

```
uv run --locked python -m bike_demand.validate --data data/raw/hour.csv
git rev-parse HEAD
uv run --locked python -m bike_demand.train --config configs/baseline.yaml
```

Code commit + canonical data ID + config + feature order

These identify the training inputs. A later report commit is a different snapshot.

Two local runs must point back to the same inputs

Use the supplied helper: SQLite stores run metadata; local files store artifacts.

RUN A – NAIVE

Actual run ID + MAE

Training mean; train/validation bounds.

RUN B – RANDOM FOREST

Actual run ID + MAE

Four settings; model version; feature order.

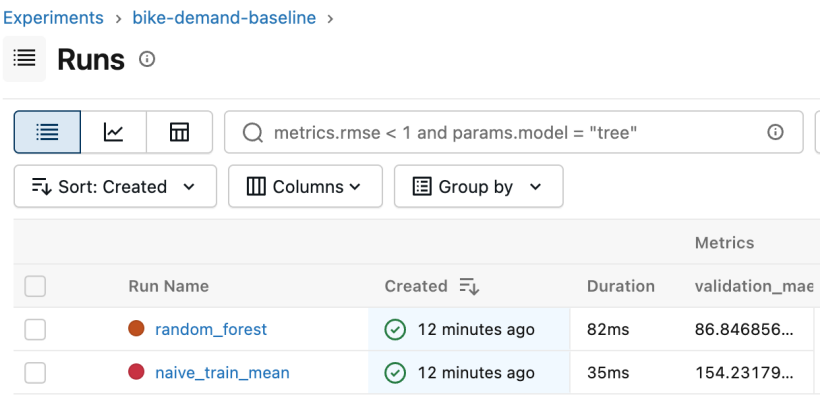
Read back

Both runs: training commit, canonical data ID, counts, bounds, predictors and settings.

Backend: `sqlite:///tracking/mlflow.db` Artifacts: `tracking/artifacts`

Find both runs in the MLflow table

Illustrative capture from a rehearsal run: your run names match, your measured MAEs will differ.



Open the Model training view; use Columns to show validation_mae. Match each run to your worksheet.

Reload the saved model, not just its screenshot

Link the local outputs to their actual run and check the returned predictions.

REQUIRED OUTPUT	OBSERVABLE CHECK
`artifacts/model.joblib`	Reload; predict finite values; recompute validation MAE.
`artifacts/model-metadata.json`	Match model version, run, training commit and data ID.
`reports/metrics.json`	Match both run IDs and MAEs.

Stretch only: repeat without changing inputs

Optional AFTER the core. Not required for m1; no additional marks.

1. **Preserve the completed first execution**

   Keep both run IDs, MAEs, settings and run-specific artifacts before repeating. Fixed output paths may be overwritten.

2. **Repeat with code, data and config unchanged**

   Record the new run IDs and observed MAEs. Compare inputs, predictions/metrics and model readback; explain any difference.

3. **Stop if core evidence is unfinished**

   Reload and review are still required. Record the stretch as not attempted rather than inventing results.

PART 3

Try to break it. Then let a peer check.

A green result is stronger when you can explain what fails and why.

CHECKPOINT C

Worked + own fault

Review the current PR

Week 1 handover

One worked fault; design the second

Two separate tiny copies of valid training rows; change only one rule in each.

1 **Fault A — worked example**

Remove temp. Assert rejection for a missing required field, then confirm the valid control passes.

2 **Fault B — your own design**

Break exactly one contract rule from page 10 or a split boundary. Retain all columns. Name the violated rule and assert the specific rejection.

3 **Keep the test isolated and the source intact**

Date faults use the splitter's boundary check. Record the actual message and a valid control; revalidate the unchanged canonical CSV.

Focused tests protect rules before the full suite runs

Valid input passes; faults fail; partitions and predictor exclusions remain correct.

TARGETED CHECKS, THEN SUPPLIED SUITE, THEN CANONICAL VALIDATION

```
uv run --locked pytest -q -m exercise
uv run --locked pytest -q
uv run --locked python -m bike_demand.validate --data data/raw/hour.csv
```

Add one test of your own for Fault B; keep the supplied tests unmodified.
Local success does not prove the PR check passed for the latest pushed commit.

Review inside the group; explain one result

Lab 2 has a different lead from Lab 1. No inter-group handover or full-class demo.

1 **Reviewer retrieves a local result**

Without verbal hints, use the worksheet to find the saved model's run, read its MAE and inspect reload evidence. Record finding + response.

2 **Check CI and local evidence separately**

Match PR checks to the current pushed revision. CI cannot read a database/model on your laptop; a green check alone does not verify local runs.

3 **Any member may answer one checkpoint question**

Why train-only mean? Why hold the test period? Where is the saved model's run ID? Point to actual evidence; merge after review and current checks.

Short reasoning; linked execution evidence

Use the Lab 2 sections of reports/week-01-team.md; do not write a second long report.

1. **Three-line summary**

   Choice: which rule did you break? Reason: what risk? Result: actual rejection/control result and any remaining limitation.

2. **Link identities and measured results**

   Training commit, data ID, config/features, counts/bounds, mean, both run IDs/MAEs and reload evidence. Repeat results only if stretch was attempted.

3. **Keep the review and ownership record**

   Link local tests and current CI results, finding + response, each member's contribution, assistance and blockers/next owner. Keep tracking DB/models out of Git.

Week 1 has a team checkpoint and one personal report

Keep project evidence and individual report submission on their separate routes.

TEAM PROJECT REPOSITORY

`m1` – ungraded

After both Week 1 labs are reviewed.
Annotated tag + short checks/blockers note.
Team code keeps peer-reviewed PRs.

YOUR PRIVATE REPORT REPOSITORY

`reports/week-1.md` – 10%

One report covers Labs 01–02.
Commit directly to personal `main`.
No report PR, CI, tag or Release.

Personal report: Sunday 23:59, using the published date and course time zone.

EXIT CHECK – EXPLAIN YOUR OWN EVIDENCE

Believable inputs. Fair comparison. Traceable results.

Why does the mean use train labels only?
Why does the test period stay held?
How does a peer find your model's actual run?

Leave with evidence, an honest limitation and a named next owner.

DATA & SPLIT

Validated source; held test.

BASELINES & TRACKING

Two runs; reload; verify.

NEXT – WEEK 2

Locked environment and container workflow.